THE EFFICIENT-AI RESEARCH TRACK

Month 2 — Building a Tiny GPT

Embeddings → Attention → a transformer you assemble yourself.
Days 31–60, self-contained.

WHERE YOU START / WHERE YOU FINISH

You start with your Month-1 text model: it predicts the next character from a fixed K-character window, and you FELT its ceiling — no memory beyond the window. You finish with a tiny GPT: a real (miniature) transformer, built from parts you hand-traced, generating noticeably better text — plus your first scaling measurement.

Prerequisites: Month-1 guide complete — especially ai_6 (backprop), ai_8 (digits), ai_10 (text). Same rules: hand-trace first, one dial at a time, commit every session.

WEEK 5 — Embeddings: Words Become Meaning

Goal: replace one-hot vectors with LEARNED vectors, and understand why that single change is the biggest quality jump per line of code in this entire track.

5.1 What's wrong with one-hot

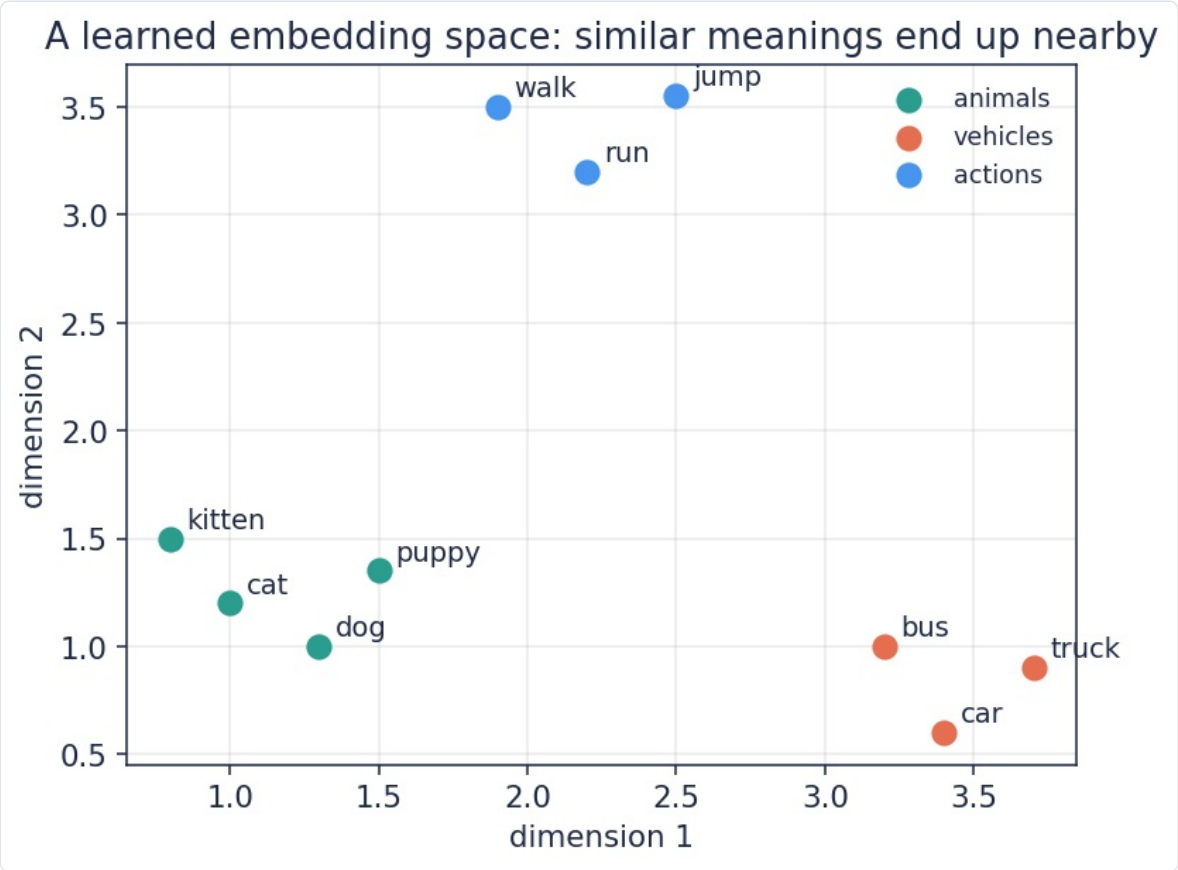
Your Month-1 model saw 'h' as `[0,0,...,1,...,0]`. Two problems hide in that:

- **Every character is equally different from every other.** The one-hot for 'e' is exactly as far from 'a' as from '?'. But that's false — vowels behave alike, punctuation behaves alike. One-hot throws that structure away, so the model must rediscover it from scratch, wasting data and training.
- **It's huge and empty.** With a vocabulary of 10,000 words, each input is 10,000 numbers, all zero except one. Almost all compute multiplies by zero.

The fix: give every symbol a short, DENSE vector of learned numbers — say 16 of them — and let training adjust those numbers like any other weight. These vectors are called **embeddings**.

```
one-hot 'cat' (vocab 10,000):  [0,0,0,...,1,...,0]        10,000 numbers, no meaning
embedding 'cat' (16 dims):     [0.21, -1.3, 0.05, ...]     16 numbers, learned meaning
```

What ends up in those 16 numbers? Whatever helps predict the next token. And something beautiful happens: symbols used in similar ways get pushed to similar vectors — because similar vectors let the network reuse the same downstream machinery for them.



After training, an embedding space organizes itself: animals cluster, vehicles cluster, actions cluster. Nobody told it these categories — usage patterns did.

WHY / FAILURE / MEASURE

Why: dense learned vectors let similar things share learning — see 'kitten' rarely, but it sits near 'cat', so cat-knowledge transfers. **Failure:** too few dimensions = words crushed together that shouldn't be; too many = wasted compute and overfitting. **Measure:** next-token accuracy vs embedding size — you'll run exactly that experiment.

5.2 The embedding layer is just a lookup

Mechanically, an embedding layer is a matrix `E` with one **row per symbol** (rows-as-meanings — your convention survives). "Embed symbol 7" = "grab row 7". That's it.

```
# ai_11_embed.py -- the embedding table
V, D = len(chars), 16              # vocab size, embedding dims
E = np.random.randn(V, D) * 0.1    # one ROW per character

def embed(ch):
    return E[char_to_id[ch]]       # a lookup, nothing more
```

How does a lookup learn? Blame flows to it like anywhere else. If row 7 was used in this example, whatever blame reaches the embedding gets applied to row 7 only (the other rows sat out — like your silent squashed neurons). Over thousands of examples, each row drifts toward whatever vector makes predictions work.

```
forward:  x = E[7]                    (grab row 7)
backward: blame_E[7] += blame_x       (only row 7 nudged this round)
```

Wire it into your Month-1 model: instead of concatenating K one-hots (K×V numbers), concatenate K embeddings (K×D numbers). With V=40 and D=16, the input shrinks 2.5x AND carries learned meaning. Retrain, compare next-char accuracy against your Month-1 baseline — same data, same H, one change. Log the before/after.

TRY IT 1

E has V=40 rows, D=16 columns. (a) How many numbers is that? (b) The letter 'q' appears 12 times in your text; roughly how many nudges does row-for-'q' receive per pass over the data, compared to row-for-'e' (3,000 appearances)? What does that imply about rare symbols?

5.3 Measure meaning: cosine similarity

Are two embeddings "close"? Use the dot product's cousin: **cosine similarity** — dot product after normalizing lengths, so only DIRECTION matters. +1 = same direction (similar), 0 = unrelated, −1 = opposite.

```
def cosine(a, b):
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

# after training, interrogate your embedding table:
for ch in "aeq.":
    sims = [(c2, cosine(E[char_to_id[ch]], E[char_to_id[c2]]))
            for c2 in chars if c2 != ch]
    sims.sort(key=lambda t: -t[1])
    print(ch, "is most like:", sims[:3])
```

Expect vowels near vowels, punctuation near punctuation. When you see it, sit with it: **the model invented phonics-ish categories from raw prediction pressure alone.** That is representation learning — arguably the deepest idea in the field, and you just watched it happen in a table you initialized as noise.

Week 5 exercises

1. Sweep D = 4, 16, 64 (one dial!). Table: next-char accuracy + parameters added. Where do returns diminish?
2. Freeze E (never nudge it) and retrain the rest. How much accuracy do learned-vs-frozen-random embeddings buy? Now you've measured what "learned meaning" is worth.
3. Efficiency lens: for V=10,000 and D=64, compare input sizes K×V vs K×D for K=8. How many multiplies did embeddings save in layer 1?

ANSWERS

T1: (a) 640. (b) ~12 vs ~3000 — rare symbols get few nudges, so their embeddings stay noisy; rare-token quality is a real problem at every scale. **1:** typically 4 is cramped, 16 fine, 64 marginal at this scale. **2:** usually several accuracy points — that gap IS the value of learning representations. **3:** 80,000 vs 512 inputs — ~156x fewer first-layer multiplies.

WEEK 6 — Attention: The Fix for the Window

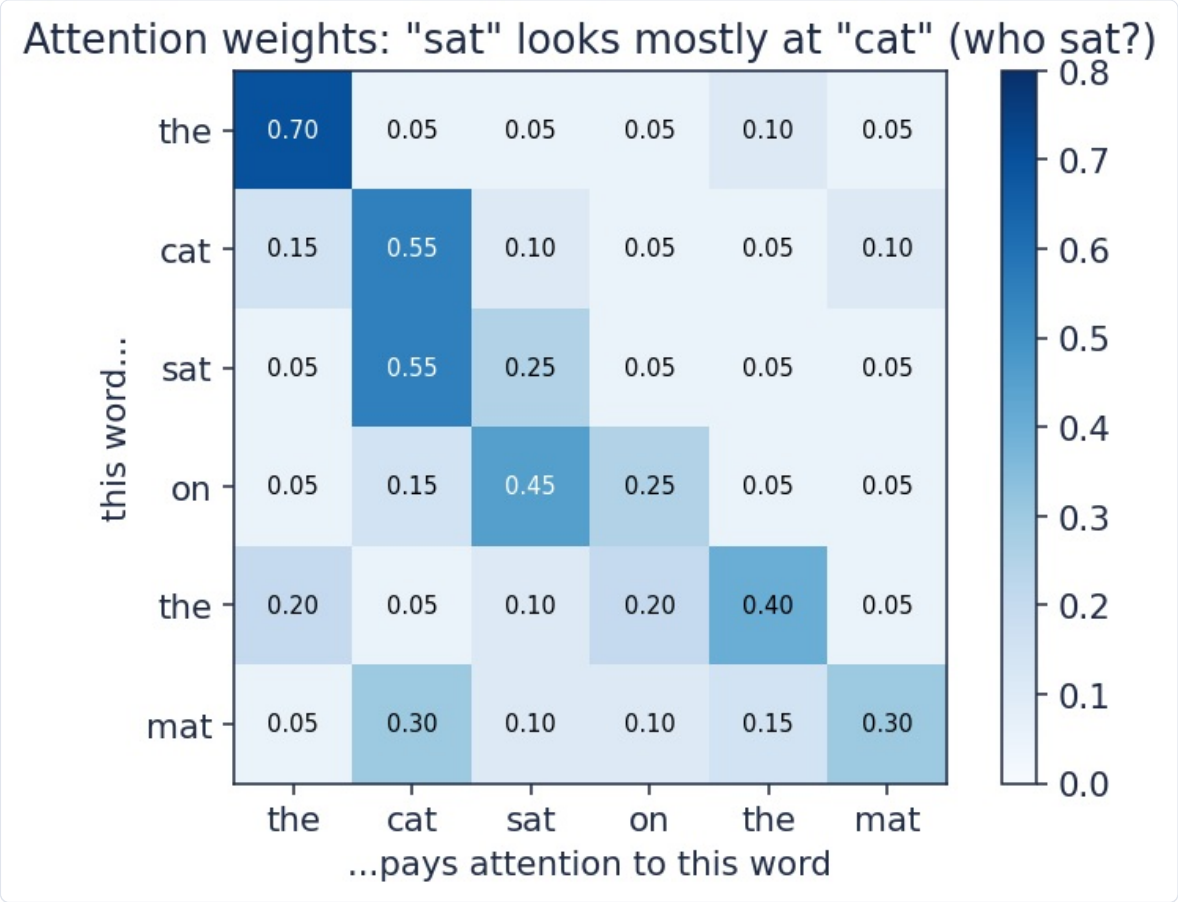
Goal: the transformer's core trick, hand-traced with tiny numbers until it's obvious. This is the week the modern era opens.

6.1 The problem attention solves

Your Month-1 model concatenated the last K characters into one flat input. Two hard limits:

- **Fixed reach:** K=8 means the model literally cannot see character 9 — "the door creaked because the ___" needs 'door', which fell off the edge.
- **Position rigidity:** "cat sat" and "sat cat" produce totally different flat inputs; the model must relearn every pattern at every offset.

Attention's idea: stop hard-wiring which past positions matter. For each position, let the model *look back over everything* and compute — per input! — how much each earlier token matters right now, then take a weighted average of their vectors. Relevance becomes dynamic, computed, learned.



Attention weights for "the cat sat on the mat": each row shows where that word looks. "sat" attends mostly to "cat" — it computed who did the sitting.

6.2 The mechanism, gently: queries, keys, values

The library analogy. Every token plays three roles, each a small learned transform of its embedding:

- **Query (Q)** — what I'm looking for. ("I'm a verb; I need my subject.")
- **Key (K)** — what I advertise. ("I'm a noun, animate, singular.")
- **Value (V)** — what I hand over if chosen. (The actual useful content.)

The recipe for one position: compare my Q against every earlier token's K (dot products — your similarity tool!), turn those match-scores into percentages, then blend everyone's V by those percentages.

```
Hand-trace, 2-dim vectors, 3 tokens; computing attention FOR token 3:
  q3 = [1, 0]
  k1 = [1, 0]   v1 = [10, 0]
  k2 = [0, 1]   v2 = [0, 10]
  k3 = [.7,.7]  v3 = [5, 5]

match scores (dot products):  q3.k1 = 1.0    q3.k2 = 0.0    q3.k3 = 0.7
to percentages (softmax):     e^1=2.72  e^0=1.00  e^.7=2.01   sum=5.73
                              w = [0.47, 0.18, 0.35]
output = 0.47*v1 + 0.18*v2 + 0.35*v3 = [4.7+0+1.75, 0+1.8+1.75] = [6.45, 3.55]
```

Read what happened: token 3's query matched token 1's key hardest (dot 1.0), so the output leans toward v1. **The output is a custom summary of the past, mixed per-token, per-input.** No fixed window did this — the match scores did.

The percentage-maker is **softmax**: exponentiate each score, divide by the sum. It's the grown-up version of your temperature sampling from Month 1 — same formula, new job. (One practical addition in code: divide scores by √d before softmax so they don't grow with vector size — big scores make softmax collapse to one winner and gradients die. It's called scaled attention; one line.)

WHY / FAILURE / MEASURE

Why: relevance becomes computed, not hard-wired — long-range connections form when needed. **Failure:** cost — every token compares with every earlier token: n tokens → ~n² comparisons. Double the context, 4x the work. **Measure:** you'll time it and see the quadratic with your own clock.

EFFICIENCY LENS

That n² is one of the most attacked costs in all of AI (linear attention, sliding windows, state-space models like Mamba exist because of it). You are one week into attention and already standing at an open research frontier — remember this feeling when you read efficiency papers later: they are all fighting this square.

TRY IT 1

Redo the hand-trace with q3 = [0, 1]. Which token wins now? Compute the new output. (Answers at week's end.)

6.3 Attention in code (yours, readable)

```
# ai_12_attention.py -- one attention head, readable style
def softmax(s):
    e = np.exp(s - s.max())         # minus max = numerical safety, same result
    return e / e.sum()

def attention(X, Wq, Wk, Wv):
    # X: one row per token (T rows, D cols) -- rows are tokens now
    Q, K, Vv = X @ Wq, X @ Wk, X @ Wv
    d = Q.shape[1]
    out = np.zeros_like(Vv)
    T = len(X)
    for t in range(T):                      # for each token...
        scores = np.full(T, -1e9)           # start: see nothing
        for j in range(t + 1):              # ...look at past+self ONLY
            scores[j] = Q[t] @ K[j] / np.sqrt(d)
        w = softmax(scores)                 # percentages
        out[t] = w @ Vv                     # blended summary
    return out
```

The `-1e9` trick implements the **causal mask**: future positions get score −infinity, so softmax gives them exactly 0 weight. A language model must not peek ahead — predicting the next token while seeing it is the leakage sin from Month 1, Week 2, in new clothing. Same principle: never let the answer touch the input.

Week 6 exercises

1. Time `attention()` for T = 50, 100, 200, 400 tokens. Plot time vs T. Is it the square you predicted?
2. Remove the causal mask and train next-token prediction. Training loss will look AMAZING. Explain, in Month-1 vocabulary, why it's a lie.
3. Print the weight matrix w for a trained sentence and find one head behaving interpretably (like the figure). Save it — first attention map in your log.

ANSWERS

T1: now q3.k2=1.0 wins; w = [0.18, 0.47, 0.35]; output = [0.18*10+0.35*5, 0.47*10+0.35*5] = [3.55, 6.45] — the mirror. **1:** yes, ~4x time per doubling. **2:** the model sees the token it's predicting — data leakage; test-time generation collapses.

WEEK 7 — Assembling the Tiny GPT

Goal: attention alone isn't a transformer. Add positions, a per-token mini-network, and two humble stabilizers — then stack. Every part earns its place.

7.1 Position: attention is order-blind

Check it: shuffle the input rows of your attention function — outputs shuffle identically but are otherwise unchanged. Attention treats text as a BAG of tokens. "dog bites man" = "man bites dog." Unacceptable.

Fix: give each POSITION its own learned embedding (a second lookup table, P — one row per position), and add it in: `X[t] = E[token] + P[t]`. Now "being word #3" is part of the vector, and attention can learn order-aware patterns. Two lookup tables, both trained by the same blame flow you know.

7.2 The transformer block: attention talks, MLP thinks

One **block** = two sub-steps:

```
step 1  ATTENTION : every token gathers a custom summary of the past   ("talk")
step 2  MLP       : each token processes its gathered info ALONE       ("think")
         -- the MLP is literally your ai_6: W2 @ squash(W1 @ x)! --
```

Then two humble one-liners that make deep stacks trainable:

- **Residual connection:** `x = x + block_output(x)`. The block only ADDS a correction to the original. Why it matters: blame flowing backward through "+" passes UNTOUCHED to earlier layers — a highway home for gradients. Without residuals, deep stacks starve their early layers (your dead-neuron discovery, at network scale).
- **LayerNorm:** before each sub-step, rescale each token's vector to mean 0, spread 1. It's your Week-2 input scaling, applied continuously INSIDE the network so signals never drift huge or tiny between layers.

```
# one transformer block, in words:
x = x + attention(norm(x))     # talk, add correction
x = x + mlp(norm(x))           # think, add correction
```

The whole tiny GPT, top to bottom:

```
token ids -> E lookup + P lookup -> block -> block -> norm -> final matrix -> V scores -> softmax -> next-token %
```

Suggested size (small enough to train on CPU in minutes, big enough to feel real): D=32, 2 blocks, 2 heads, context T=32, vocab = your characters. That's `ai_13_tinygpt.py`. Assemble it from YOUR parts: embedding (ai_11), attention (ai_12), MLP (ai_6), plus the four one-liners above. You will have built a transformer from scratch, no framework, every line traceable.

REALITY CHECK

Backprop through all of this by hand-written code is fiddly — blame through softmax and layernorm is the hardest bookkeeping in the track. Budget a full session for debugging; use tiny T (3 tokens) and check every shape. If a gradient bug eats two evenings, that's normal and the debugging IS the education. (Professionals use autograd for exactly this reason — we go manual ONCE, for X-ray vision, then earn the right to frameworks in Month 3+.)

Week 7 exercises

1. Ablation study (a real research method): train tiny GPT (a) full, (b) no position table, (c) no residuals, (d) MLP removed. One table: final loss each. Which amputation hurt most? Hypothesis first!
2. Count your GPT's parameters by hand (E + P + per-block Wq,Wk,Wv,Wo + MLP + final matrix). Verify with .size sums. Then per-token multiplies. Your ai_9 habit, upgraded.

ANSWERS

1: typically (c) no-residuals hurts most at depth (training barely moves), (b) no-positions produces bag-of-text babble, (d) no-MLP limps but works — attention alone is surprisingly strong. **2:** depends on your sizes; the point is the habit — expect tens of thousands of parameters.

WEEK 8 — Train It Right + Your First Scaling Study

Goal: the practical craft of training the tiny GPT — then the month's capstone: a miniature scaling law, measured by you.

8.1 The proper wrongness meter for next-token prediction

Your model outputs V percentages. The target is one correct character. **Cross-entropy** is the standard meter: `loss = -log(probability the model gave the RIGHT answer)`.

```
model says correct char had probability 0.90  ->  loss = -log(0.90) = 0.105  (barely wrong)
model says correct char had probability 0.10  ->  loss = -log(0.10) = 2.303  (very wrong)
model says correct char had probability 0.01  ->  loss = -log(0.01) = 4.605  (confidently wrong = punished HARD)
```

That last line is why cross-entropy beats MSE here: it savages confident wrongness. Bonus 1: its blame signal is beautifully simple — `blame_scores = probabilities - one_hot(target)` (cleaner than MSE's!). Bonus 2: e^loss has a meaning — "the model is as unsure as choosing among e^loss options." Loss 2.0 ≈ picking from ~7.4 chars. Watch that number fall from ~vocab-size toward small: uncertainty literally shrinking.

8.2 Batching: many examples per nudge

Month 1 nudged after every single example — noisy, like adjusting your route after every pothole. Now: run 16–32 windows, AVERAGE their blames, nudge once. Smoother steps, and the matrix math does all examples in one multiply (the vectorization from your Stage-0 guide, finally cashing in). Add momentum from Month-1 Week 3. This is 90% of what industrial training loops do.

8.3 Read the curves like a doctor

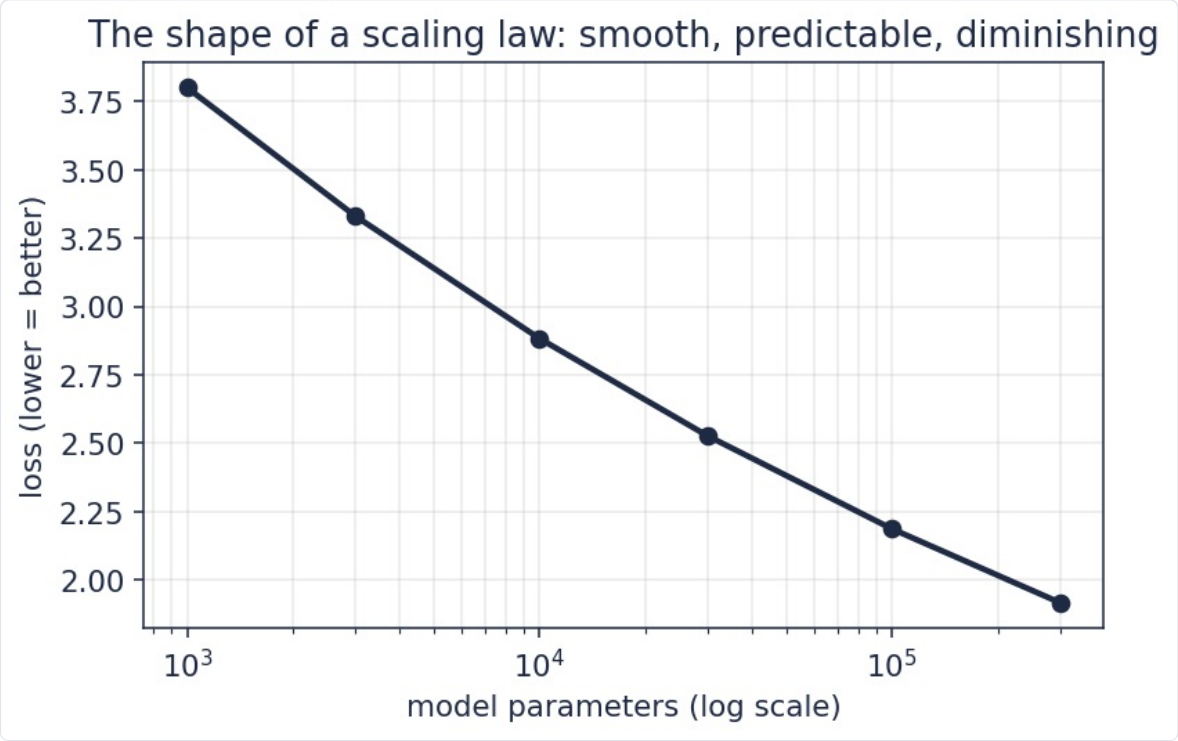
Symptom	Likely cause	First fix
Loss flat from the start	step too small, or a blame-path bug	overfit ONE batch first — if you can't hit ~0 loss on 32 examples, it's a bug, not a size problem
Loss explodes / NaN	step too big; scores blowing up	lower step 10x; check the √d scaling
Train falls, val rises	overfitting (old friend)	more data, or early stop (Month-1 W3)
Both fall then plateau	model at capacity	this is REAL — it's what scaling laws describe; see 8.4

The "overfit one batch" trick is the single most useful debugging move in deep learning: it separates "my code is broken" from "my model is small" in five minutes.

8.4 Capstone: your miniature scaling law

The famous result: loss falls smoothly and predictably as a POWER LAW in model size / data / compute. Frontier labs bet billions on those curves. You'll draw one for real:

1. Fix the data. Train 4 models: D=8 / 16 / 32 / 64 (scale block width with it), same recipe, to the same number of steps.
2. Record best validation loss + parameter count for each.
3. Plot loss vs parameters with the x-axis on a log scale.



The expected shape: near-straight on a log axis. Smooth, predictable, diminishing — the curve the whole field is built on.

If your four dots trace that shape, you have reproduced — in miniature and with all honest caveats about scale — the empirical backbone of modern AI. Write the Month-2 `writeup.md`: question, setup, the plot, surprises, limits. Then generate a page of text from your best GPT next to your Month-1 model's output. Read

them side by side. That difference is what you built this month.

EFFICIENCY LENS — THE BRIDGE TO MONTH 3

Your scaling curve says: quality costs parameters. Month 3 is the counterattack — getting the quality WITHOUT the parameters: cleaner data (better food beats more food), distillation (a teacher compresses knowledge into a student), quantization and low-rank tricks (store the knowledge cheaper). Every tool you now own gets pointed at that curve.

Week 8 exercises

1. Two temperatures deserve science: generate 5 samples each at t = 0.5 / 0.9 / 1.3 from the SAME model; have a friend (or future-you) rank readability blind. Log the winner.
2. Hold parameters fixed; scale DATA instead: train your D=32 model on 25% / 50% / 100% of your text. Plot loss vs data. Which bought more at your scale — parameters or data? (Write your guess first.)
3. Estimate: using your per-token multiplies from Week 7 ex.2 and your steps×batch×T count, how many total multiplies did your best training run cost? Keep that number — Month 3 tries to beat it.

ANSWERS

1: usually 0.9-ish wins; extremes lose for opposite reasons — connect to the weighted-die picture. **2:** at tiny scale, data usually wins (models are data-starved); note that "which is scarce?" is exactly the Chinchilla question. **3:** typically hundreds of millions to billions — and frontier runs are ~10^{15} times bigger; keep the ratio in your log.

Month 2 closing checklist

- ☐ Week 5: my model uses learned embeddings, and I've SEEN similar characters cluster (cosine tool)
- ☐ Week 6: I can hand-trace attention for 3 tokens cold, and I timed the n^2 myself
- ☐ Week 7: my tiny GPT is assembled from my own parts, and my ablation table says what each part is worth
- ☐ Week 8: cross-entropy + batching run clean, and my 4-dot scaling plot is in the repo with a write-up

Codebase now: ai_11_embed.py → ai_12_attention.py → ai_13_tinygpt.py → ai_14_train.py + scaling_writeup.md, tagged v0.4.